SCGIP — Penetration Test Report (Production)

Target: `https://sanitation-co-governance-platform-ntt71eypz.vercel.app` **Date:** 2026-09-25 · **Authorization:** Owner-directed test of owner's own application **Tester:** Manual verified test-suite (curl-based, evidence captured) + Strix v1.6.2 multi-agent scan (attempted — see Addendum) **Stack:** Next.js 16 (App Router) · Prisma 6 · Neon PostgreSQL · custom HMAC session auth · Vercel

---

Executive Summary

The application's **authentication and object-level authorization are solid**: role injection, IDOR across every complaint action, forged sessions, and unauthenticated access to sensitive APIs were all correctly blocked.

The headline finding is a **critical-by-design flaw in the corporation webhook**: `/api/authority/webhook` performs **no authentication at all** in production because its shared-secret check *fails open* when `AUTHORITY_WEBHOOK_SECRET` is unset — which it is. Any internet user can set any complaint to any status (escalate, "resolve", reject), falsify SLA analytics, and trigger notifications to real users. Full working proof-of-concept below.

#	Severity	Finding	Status
F-01	**HIGH**	Authority webhook fail-open: unauthenticated complaint status control	Confirmed w/ PoC
F-02	MEDIUM	No rate limiting / lockout on login (brute-force possible)	Confirmed
F-03	MEDIUM	Session cookie missing `Secure` flag	Confirmed
F-04	MEDIUM	AI chatbot API unauthenticated (abuse/spoofing surface)	Confirmed
F-05	LOW	User enumeration via registration (409 on existing email)	Confirmed
F-06	LOW	Session token has no internal expiry (7-day cookie only)	Confirmed (code)

Zero High/Medium issues found in: authentication logic, IDOR/authorization on all complaint APIs, role management, injection handling, security headers, or secret handling.

Confirmed Findings

F-01 · HIGH — Authority webhook is unauthenticated (fail-open secret check)

Location: `app/api/authority/webhook/route.ts`

```
const secret = process.env.AUTHORITY_WEBHOOK_SECRET || ""
if (secret && request.headers.get("X-Webhook-Secret") !== secret) {
  return Response.json({ error: "Forbidden" }, { status: 403 })
}
```

When `AUTHORITY_WEBHOOK_SECRET` is **not configured**, `secret` is `""`, the `secret && …` guard short-circuits, and the handler proceeds with **zero authentication**. This is the production state (verified).

Impact: Anyone on the internet can, for any complaint ID: - set arbitrary status (`Closed` , `Rejected` , `Escalated` , …) → falsified resolutions, suppressed complaints - poison SLA/SERI analytics and dashboards used for governance reporting - trigger notification spam to real citizens/officers - fabricate the audit/transition trail (`actor: "authority-gateway"`)

Proof of Concept (performed on a self-created test complaint, then restored):

```
POST /api/authority/webhook   (no cookies, no headers)
{"complaint_id":"cmuh828pv0001jp04jp0r50ht","status":"Escalated"}
→ 200 {"ok":true}          status observed: Pending → Escalated

POST /api/authority/webhook (no cookies, no headers)
{"complaint_id":"cmuh828pv0001jp04jp0r50ht","status":"Pending"}
→ 200 {"ok":true}          status restored

Audit trail captured: "Authority webhook: status set to Escalated" ×2, "…Pending" ×2
```

Fix (one line — fail *closed*):

```
const secret = process.env.AUTHORITY_WEBHOOK_SECRET
if (!secret || request.headers.get("X-Webhook-Secret") !== secret) {
  return Response.json({ error: "Forbidden" }, { status: 403 })
}
```

…then set `AUTHORITY_WEBHOOK_SECRET` in Vercel (strong random value) and share it with the corporation integration. Until the integration is real, the endpoint should 403 unconditionally.

F-02 · MEDIUM — No rate limiting or account lockout on login

Evidence: 6 rapid consecutive wrong-password attempts against `/api/auth/login` returned `401` every time — no throttling, no delay, no lockout, no captcha. Vercel's platform does not add rate limiting for you.

Impact: Online password brute-force / credential stuffing is unthrottled. Admin account is a high-value target.

Fix (pick any): middleware-based limiter keyed on IP+email (e.g. Upstash Redis free tier), progressive delay after N failures, temporary lockout, or Vercel WAF rate rule on `POST /api/auth/login` .

F-03 · MEDIUM — Session cookie missing `Secure` attribute

Evidence (live Set-Cookie from production login):

```
Set-Cookie: scgip_session=…; Path=/; Expires=…; Max-Age=604800; HttpOnly; SameSite=lax
```

`HttpOnly` ✅, `SameSite=lax` ✅, but **no** `Secure` — browsers may transmit the session cookie over plain HTTP (e.g. MITM downgrade, typed http:// URL).

Fix: add `secure: true` in `lib/server/auth.ts` `setSessionCookie()` (site is HTTPS-only + HSTS, so no downside).

F-04 · MEDIUM — AI chatbot endpoint is unauthenticated

Evidence: `POST /api/chat` returns full assistant answers with no session (knowledge-base mode; on prod the FastAPI sidecar is absent so no LLM cost is incurred, but the route also proxies to `FASTAPI_URL` when reachable).

Impact: Anonymous abuse surface: scripted scraping of the FAQ engine, spam through the streaming endpoint, and — if a paid LLM sidecar is ever attached — **unbilled LLM cost from anonymous users**.

Fix: require a session (any role) for `POST /api/chat`; keep `GET /api/chat` (health) public if the UI needs it.

F-05 · LOW — User enumeration via registration

Evidence: registering an existing email returns `409 {"error":"Email already registered"}`.

Impact: attacker can enumerate which emails have accounts. Standard low-severity issue.

Fix (optional): return a generic success-ish response + email the real user, or accept the risk (typical for civic platforms).

F-06 · LOW — Session token contains no expiry

Location: `lib/server/auth.ts` — token payload is `{email, iat}` HMAC-signed; verification checks signature only, never `iat` age. The 7-day limit comes solely from the cookie's `Max-Age`.

Impact: a copied cookie value stays valid forever even after the cookie "expires" in the browser.

Fix: in `getCurrentUser()`, reject tokens older than 7 days: `if (Date.now() - iat > 7*24*3600*1000) return null`.

Tested & Passed ✅

Check	Result
Role injection at registration (`role:"admin"` , `isAdmin:true` in body)	✅ Ignored — account forced to `citizen`
Citizen → `/api/admin/users` (list/create staff)	✅ 401
Citizen → admin-only status flip	✅ 401
Citizen reading another user's complaint (IDOR)	✅ 404 (no existence oracle)
Citizen rating/confirming another user's complaint	✅ 403
Citizen complaint list scoping	✅ Own records only (0 visible of others')
Forged HMAC session cookie (wrong signature)	✅ Rejected → anonymous
NoSQL/operator injection in login JSON (`$gt` , `$ne`)	✅ Normal 401 (Prisma string-casts)
Unauthenticated access to complaints / notifications / SSE stream / import	✅ 401
Security headers: HSTS(+preload), CSP (incl. tile.openstreetmap.org), XFO DENY, nosniff, Referrer-Policy, Permissions-Policy	✅ All present
Registration role hardcoding (server-side `role:"citizen"`)	✅ Confirmed
Session flags `HttpOnly` + `SameSite=lax`	✅ Present (but see F-03)

Not Tested / Out of Scope

- **Stored XSS payloads were NOT injected** into the production complaint DB (would pollute real user data). JSX auto-escaping reviewed in code instead — no `dangerouslySetInnerHTML` in complaint rendering paths. Recommend testing XSS on a staging/local copy.
- Vercel/Neon platform-level attack surface (out of application scope).
- DoS, flooding, or high-volume fuzzing (explicitly excluded to protect real users on prod).
- The FastAPI agenticAI sidecar (not deployed to production).

Remediation Priority

1. **Now:** F-01 webhook fail-closed fix + set `AUTHORITY_WEBHOOK_SECRET` (one line + one env var).
2. **This week:** F-03 `secure: true` cookie, F-02 login rate limiting.
3. **Next:** F-04 chat auth gate, F-06 token expiry, F-05 (optional).

Addendum — Strix v1.6.2 Automated Scan

- Mode: `quick` , non-interactive, authenticated (admin creds supplied via instruction file), target = prod URL, sandbox `ghcr.io/usestrix/strix-sandbox:1.3.0` (Docker).
- Multi-agent pipeline started correctly (Root Agent → Reconnaissance Agent), httpx reconnaissance data captured against the target.

- **0 findings produced**: Google free-tier Gemini endpoint returned sustained mid-stream `503 ServiceUnavailableError` (`litellm.MidStreamFallbackError`), 139 stream retries across two runs; the run made partial progress but could not converge within the free quota.
- All report findings therefore come from the **manual verified suite** (evidence above). Rerun guidance: off-peak hours, or configure Strix with a paid/Groq provider key, then `strix -t <url> -m standard -n --resume <run>`.

Secrets handling note: admin credentials were read from the gitignored `.env.deploy-secrets` file at runtime and were never written into this report, command logs, or scan instructions beyond the ephemeral local instruction file (`~/.scgip-strix-instruction.txt` — safe to delete).